# Hafta 3 — Derinlemesine

> ## ⚠️ BU DOSYA SINAV KAPSAMI DIŞINDADIR
>
> Buradaki hiçbir konudan sınavda soru sorulmaz. Bu dosya, "*peki bu nasıl çalışıyor?*" diye soran öğrenci içindir. Sınav kapsamı `H03-ders-notu.md` dosyasıdır.
>
> Dönem projesinde işinize yarayacak birkaç şey de buradadır (bölüm 3, bölüm 6).

---

## 1. `.cshtml` Hangi C# Sınıfına Dönüşür?

Ders notu bölüm 1.1'de "*Razor dosyası önceden derlenmiş bir C# sınıfıdır*" dedik. Bu, mecaz değil; gerçekten bir sınıf üretilir.

Şu görünüm:

```
@model List<KampusKitap.Models.Kitap>
<h1>Kitaplar</h1>
@foreach (var k in Model)
{
    <li>@k.Baslik</li>
}
```

kabaca şuna dönüşür:

```
public class Views_Kitap_Index : RazorPage<List<KampusKitap.Models.Kitap>>
{
    public override async Task ExecuteAsync()
    {
        WriteLiteral("<h1>Kitaplar</h1>\n");
        foreach (var k in Model)
        {
            WriteLiteral("<li>");
            Write(k.Baslik);          // ← HTML KODLAYAN yazım
            WriteLiteral("</li>\n");
        }
    }
}
```

Buradan üç şey okunur:

| Gözlem | Sonucu |
|---|---|

| | |
|---|---|
| `WriteLiteral` ile `Write` **farklı** metotlar | `WriteLiteral` ham HTML basar (sizin yazdığınız etiketler); `Write` **kodlar** (veri) |
| `@Html.Raw(x)` → `Write(x)` yerine `WriteLiteral`-benzeri davranış | Kodlama bu yüzden atlanır (ders notu bölüm 1.5) |
| Sınıf `RazorPage<TModel>` türetir | `Model`, `ViewData`, `Html`, `Url` bu taban sınıftan gelir |

`@{ }` blokları metot gövdesine olduğu gibi yazılır; `@if` / `@foreach` ise gövdeyi `WriteLiteral` çağrılarıyla bölerek C# denetim yapısına dönüşür. Razor'un "`@`'tan sonrası C#" kuralı, aslında **parser'ın HTML modu ile C# modu arasında geçiş yapmasıdır**.

---

## 2. Derleme Zamanı mı, Çalışma Zamanı mı?

ASP.NET Core'da `.cshtml` dosyaları **derleme (build) ve yayın (publish) zamanında** Razor SDK tarafından derlenir. Yani `dotnet publish` çıktısında görünümleriniz artık `.cshtml` metni değil, derlenmiş koddur.

**Çalışma zamanı derleme (runtime compilation)** ayrı bir pakettir (`Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation`) ve dosyayı siz çalışırken değiştirdiğinizde yeniden derler.

> ⚠️ **.NET 10'a özel: runtime compilation artık obsolete**
>
> Microsoft, Razor **çalışma zamanı derlemesini .NET 10 ile birlikte kullanım dışı (obsolete)** ilan etti. Gerekçeleri:
>
> - **.NET Hot Reload**'u devre dışı bırakır
> - `global using` ve örtük (implicit) `using` yönergelerini desteklemez
> - Blazor bileşenlerinde desteklenmez
> - Üretim ortamı için önerilmez
>
> **Bugünün doğru yolu:** geliştirmede **Hot Reload**, üretimde **derleme zamanı derleme**. İnternette bulacağınız "görünümleri canlı düzenlemek için AddRazorRuntimeCompilation ekleyin" tavsiyesi .NET 10 için **eskimiştir**.

Pratik sonuç: derste `.cshtml` değişikliğini görmek için çoğu zaman `dotnet watch` yeterlidir; görmüyorsanız uygulamayı yeniden başlatın.

Kaynak: Razor file compilation in ASP.NET Core

---

## 3. Görünüm Arama Sırasını Değiştirmek

Ders notu bölüm 2.5'te iki yol gördük. Bu sıra sabit değildir; `IViewLocationExpander` ile genişletilebilir. Tipik kullanım: çok kiracılı (multi-tenant) bir uygulamada her müşteriye kendi görünümünü vermek.

```
public class TemaGenisletici : IViewLocationExpander
{
    public void PopulateValues(ViewLocationExpanderContext context)
    {
        context.Values["tema"] = context.ActionContext.HttpContext.Request.Host.Host;
    }

    public IEnumerable<string> ExpandViewLocations(
        ViewLocationExpanderContext context, IEnumerable<string> viewLocations)
    {
        var tema = context.Values["tema"];
        // Önce temaya özel yol, sonra varsayılanlar
        return new[] { $"/Temalar/{tema}/{{1}}/{{0}}.cshtml" }.Concat(viewLocations);
    }
}
```

```
builder.Services.Configure<RazorViewEngineOptions>(o =>
    o.ViewLocationExpanders.Add(new TemaGenisletici()));
```

Yer tutucular: `{0}` görünüm adı, `{1}` controller adı, `{2}` alan (area) adı.

> 💡 Ders notundaki hata mesajının listelediği yollar, bu genişleticilerden **sonra** oluşan nihai listedir. Yani bir genişletici eklediyseniz, hata mesajı onu da gösterir.

---

## 4. `ViewComponent`'in Döndürebileceği Sonuçlar

Ders notu bölüm 3.2'de yalnızca `View(...)` kullandık. Taban sınıfın üç yardımcısı vardır:

| Dönüş | Ne yapar | Ne zaman |
|---|---|---|
| `View()` / `View(model)` | `Default.cshtml`'i render eder | Olağan durum |
| `View("Kompakt", model)` | Aynı klasörde `Kompakt.cshtml` | Aynı bileşenin iki görünümü |
| `Content("metin")` | **Kodlanmış** düz metin | Tek satırlık çıktı; görünüm dosyasına değmez |

```
public IViewComponentResult Invoke(bool kompakt)
    => View(kompakt ? "Kompakt" : "Default", _sayac.Oku());
```

**`Invoke` mi `InvokeAsync` mi?** Çatı ikisini de bulur. Kural basittir: içeride `await` edeceğiniz bir iş varsa (veritabanı, HTTP) `InvokeAsync`, yoksa `Invoke`. Asenkron olmayan bir işi `async` yapmak hiçbir şey kazandırmaz.

**View Component bir controller değildir:**

- Route'lanmaz, adresi yoktur, doğrudan HTTP isteği alamaz

- Model binding ve filtre (filter) hattından geçmez
- `Request`'e erişebilir, ama ona karar verdirmeyin — parametre alması daha iyidir

---

## 5. `PartialAsync` ve `RenderPartialAsync` Farkı

Üç yazım da aynı partial'ı çağırır:

```
<partial name="_KitapKart" model="k" />              @* Tag Helper — tercih edilen *@
@await Html.PartialAsync("_KitapKart", k)            @* sonucu DÖNDÜRÜR *@
@{ await Html.RenderPartialAsync("_KitapKart", k); } @* doğrudan çıktıya YAZAR *@
```

| | `PartialAsync` | `RenderPartialAsync` |
|---|---|---|
| Sonuç | `IHtmlContent` **döndürür** | Doğrudan yanıt akışına **yazar** |
| Yazım | `@await …` | `@{ await …; }` — süslü parantez **zorunlu** |
| Bellek | Ara tampon oluşur | Oluşmaz, biraz daha hızlı |
| Değişkene atanabilir mi | Evet | Hayır |

Fark ölçülebilir olmak için binlerce yineleme gerekir. **Tag Helper yazımını kullanın**; diğer ikisi eski kodda karşınıza çıkacağı için burada.

> ⚠️ `@Html.Partial(...)` (senkron, `await` siz) sürümü **kullanılmamalıdır** — asenkron bağlamda kilitlenmeye (deadlock) yol açabildiği için analiz uyarısı verir.

---

## 6. `_ViewStart` Zinciri ve Çalışma Sırası

Birden fazla `_ViewStart.cshtml` varsa hepsi çalışır, **kökten yaprağa doğru**:

```
Views/_ViewStart.cshtml             ← önce
Views/Kitap/_ViewStart.cshtml       ← sonra (öncekini EZEBİLİR)
Views/Kitap/Index.cshtml            ← en son
```

Yani `Views/Kitap/_ViewStart.cshtml` içinde `Layout = "_KitapLayout";` yazarsanız, yalnızca o klasördeki görünümler farklı layout kullanır. Kökteki dosya yine çalışır — ama değeri ezilir.

`_ViewImports.cshtml` için de aynı zincir geçerlidir, bir farkla: yönergeler **birikir** (`@using`'ler toplanır), ezilmez. İstisna `@model` ve `@inherits`: onlarda en yakın olan kazanır.

Katmanlı bir layout da kurulabilir — layout'un kendi layout'u olabilir:

```
@* Views/Shared/_AdminLayout.cshtml *@
@{
    Layout = "_Layout";        @* site layout'unun içine yerleşir *@
}
<div class="admin-kenar">…</div>
@RenderBody()
```

---

## 7. `ViewData` / `ViewBag` ile Layout'a Veri Taşımak

Layout, görünümün modelini bilmez (bilmemelidir de). Ortak alan `ViewData`'dır:

```
@* Görünümde *@
@{
    ViewData["Title"] = "Kitaplar";
}
```

```
@* Layout'ta *@
<title>@ViewData["Title"] — KampusKitap</title>
```

`ViewBag`, `ViewData`'nın `dynamic` kabuğudur: `ViewBag.Title` ile `ViewData["Title"]` **aynı sözlüğe** yazar. `dynamic` olduğu için yazım hatası derlemede yakalanmaz.

> SOON `ViewData`, `ViewBag` ve `TempData` üçlüsü **Hafta 4 bölüm 1**'de — ömür farkı ve karar kuralıyla birlikte — ayrıntılı işlenir. Burada yalnızca layout'a başlık taşıdığımız için var.

Daha sağlam yol, layout'un kendi modelini bir View Component ile beslemektir — bu hafta öğrendiğiniz nedenle: **veriyi kendi getirir**.

---

## 8. HTML Kodlaması: Bağlam Önemlidir

Ders notu bölüm 1.5'te Razor'ın otomatik kodlamasını gördük. Eksik kalan nokta şu: **kodlama bağlama göre değişir.**

| Veri nereye giriyor | Doğru kodlama | Razor otomatik yapar mı |
|---|---|---|
| HTML gövdesi (`<p>@x</p>`) | HTML encoding | ✅ evet |
| HTML özniteliği (`title="@x"`) | HTML attribute encoding | ✅ evet |
| **JavaScript içi** (`<script>var a = "@x";</script>`) | JavaScript encoding | ❌ **hayır — tehlikeli** |
| URL sorgu dizesi | URL encoding | ❌ hayır |

Üçüncü satır gerçek bir tuzaktır. Doğru yol, değeri **bir veri özniteliğine** koyup JavaScript'te okumaktır:

```
<div id="veri" data-baslik="@Model.Baslik"></div>
<script>
  const baslik = document.getElementById("veri").dataset.baslik;
</script>
```

Kod içinde kodlayıcılara DI ile de erişilebilir: `HtmlEncoder`, `JavaScriptEncoder`, `UrlEncoder`.

### Türkçe karakterler neden `ç` görünüyor?

Kendi projenizde **Ctrl+U** yapınca şöyle bir şey görürsünüz:

```
<p>Otostop&#xE7;unun Galaksi Rehberi</p>     <!-- veri: @Model.Baslik -->
<p>İletişim: baran.kaynak@ornek.edu.tr</p>   <!-- .cshtml'de düz yazdığınız metin -->
```

İkisi de Türkçe, biri kodlanmış diğeri değil. Sebep bölüm 1'deki ayrımdır:

| Kaynak | Nasıl yazılır | Kodlanır mı |
|---|---|---|
| `.cshtml`'de **düz yazdığınız** HTML ve metin | `WriteLiteral` | ❌ hayır — olduğu gibi gider |
| `@ifade` ile yazdırdığınız **veri** | `Write` | ✅ evet |

Kodlayıcının "güvenli listesi" varsayılan olarak **Basic Latin** ile sınırlıdır; bu aralığın dışındaki her karakter (ç, ğ, ş, ı, İ, Çince, Kiril...) sayısal kaçışa çevrilir. **Bu bir hata değildir** — tarayıcı `ç` gördüğünde ekrana `ç` çizer, kopyalarsanız `ç` gelir, arama motoru doğru okur.

Rahatsız ediyorsa güvenli liste genişletilebilir:

```
builder.Services.AddSingleton<HtmlEncoder>(
    HtmlEncoder.Create(allowedRanges: new[] {
        UnicodeRanges.BasicLatin, UnicodeRanges.Latin1Supplement,
        UnicodeRanges.LatinExtendedA }));
```

> ⚠️ Bu ayar yalnızca **DI ile alınan** kodlayıcıyı etkiler. `HtmlEncoder.Default`'a doğrudan erişirseniz yine Basic Latin listesi kullanılır.

---

## 8b. İleri Yönergeler

Ders notu bölüm 1.8 bu haftanın yönergelerini listeliyor. Nadiren gerekenler:

| Yönerge | Ne yapar |
|---|---|
| `@inherits BaseView<TModel>` | Üretilen sınıfın taban sınıfını değiştirir; `@model` yerine geçer |
| `@namespace Benim.Alanim` | Üretilen sınıfın namespace'ini belirler (klasör yapısına göre türetilir) |
| `@implements IDisposable` | Üretilen sınıfa arayüz ekler |

| | |
|---|---|
| `@attribute [Authorize]` | Üretilen sınıfa öznitelik ekler — **Hafta 11**'de işimize yarayacak |
| `@preservewhitespace true` | Çıktıdaki boşlukları korur (ağırlıklı olarak Blazor) |

`@inherits` ile `@model` **aynı anda** kullanılırsa `@model`, `@inherits` ile verilen taban sınıfın generic parametresini belirler. Bu ikisinin birlikte doğru çalışması için taban sınıfın `RazorPage<TModel>` türetmesi gerekir.

> **Kural:** bu yönergelerin hiçbirine dönem projesinde ihtiyacınız olmayacak. Burada olma sebepleri, başka bir projede karşınıza çıktığında tanıyabilmeniz.

---

> **Kodlama çıktı anında yapılır, veritabanında saklanmaz.** Kodlanmış veriyi saklarsanız aramak, karşılaştırmak ve başka bir bağlamda (e-posta, PDF) kullanmak bozulur.

Kaynak: Prevent XSS in ASP.NET Core

---

## 9. Razor Pages ile İlişkisi

Bu derste **MVC** kullanıyoruz: controller + görünüm. ASP.NET Core'un ikinci bir sayfa modeli daha var: **Razor Pages**.

| | MVC | Razor Pages |
|---|---|---|
| Dosya | `Controllers/KitapController.cs` + `Views/Kitap/Index.cshtml` | `Pages/Kitap/Index.cshtml` + `Index.cshtml.cs` |
| Yönlendirme | Route kalıbı | **Dosya yolu** doğrudan URL |
| Kod nerede | Controller'da, action'larda | Sayfanın yanındaki `PageModel` sınıfında |

**Ortak olan:** Razor sözdizimi, Layout, `_ViewStart`, `_ViewImports`, partial, View Component. Yani bu haftanın öğrendiklerinin **tamamı** Razor Pages'te de aynen geçerlidir.

> Bu dersin MVC'yi seçme gerekçesi `H02-detay.md` 'dedir: sınav ve EBS izlencesi MVC üzerine kuruludur, ayrıca controller/action ayrımı Hafta 12'deki Web API'ye doğrudan köprü kurar.

---

## 10. Daha İleri Gitmek İsteyenlere

| Konu | Nerede |
|---|---|
| Razor sözdizimi — tam referans | https://learn.microsoft.com/aspnet/core/mvc/views/razor?view=aspnetcore-10.0 |
| Görünüm derlemesi ve runtime compilation'ın durumu | https://learn.microsoft.com/aspnet/core/mvc/views/view-compilation?view=aspnetcore-10.0 |

| | |
|---|---|
| View component — tüm senaryolar | https://learn.microsoft.com/aspnet/core/mvc/views/view-components?view=aspnetcore-10.0 |
| Partial view — üç çağırma biçimi | https://learn.microsoft.com/aspnet/core/mvc/views/partial?view=aspnetcore-10.0 |
| XSS — kodlayıcıları özelleştirmek | https://learn.microsoft.com/aspnet/core/security/cross-site-scripting?view=aspnetcore-10.0 |
| Razor class library (görünümleri paketlemek) | https://learn.microsoft.com/aspnet/core/razor-pages/ui-class?view=aspnetcore-10.0 |

> ⚠️ Bağlantıların sonundaki `?view=aspnetcore-10.0` **kasıtlıdır.** Sürüm sabitlenmezse arama motoru sizi eski sürüme düşürür ve `Startup.cs` 'li örneklerle karşılaşırsınız.